

Práctica 1. Estructuras lineales

1. Introducción

Yontendo, la conocida empresa de desarrollo de videojuegos, planea lanzar su nueva consola **Super Yontendo** a principios del próximo año. Como parte de este lanzamiento, se encuentra desarrollando una nueva entrega de su exitosa serie *Super Marco Bros.*, en la que nuestros oficinistas favoritos, **Marco** y **Luis**, deberán rescatar nuevamente a la princesa **Mango** de las garras del malvado **Licenciado Brouser**.

Como parte del desarrollo, se propone diseñar un nuevo motor para la construcción y administración de los escenarios del juego. En particular, se requiere almacenar los enemigos presentes en el escenario utilizando una estructura, colección o TDA que permita localizarlos y eliminarlos conforme Marco o Luis avancen.

Tu trabajo como desarrollador consiste en implementar una **prueba de concepto del motor del juego utilizando Java y el paradigma orientado a objetos**.

2. Descripción del escenario

Un escenario está formado por una secuencia de **bloques** sobre los cuales puede desplazarse el personaje. Cada bloque se identifica mediante su posición dentro del escenario, comenzando en la posición 0.

El tamaño del escenario es variable y deberá especificarse en el momento de crear el objeto correspondiente.

Cada escenario deberá contener:

- Un personaje, que podrá ser inicialmente **Marco** o **Luis**.
- Una colección de enemigos distribuidos en diferentes posiciones del escenario.
- La información necesaria para representar el tamaño y el estado actual del escenario.

Para simplificar esta prueba de concepto, el personaje únicamente podrá desplazarse **en una dirección y sobre un solo eje**, avanzando desde la posición inicial hacia el final del escenario.

3. Personajes

Cada personaje deberá almacenar, como mínimo, la siguiente información:

- Un identificador que permita determinar si se trata de Marco, Luis o de un personaje que pudiera incorporarse posteriormente.
- El número de vidas disponibles.
- Su posición actual dentro del escenario.

Al crear un escenario, el personaje deberá colocarse inicialmente en el bloque correspondiente a la posición 0.

En esta etapa del desarrollo, todos los personajes serán **invencibles**. Por lo tanto, cuando un personaje llegue o pase por una posición en la que se encuentre un enemigo, dicho enemigo deberá considerarse eliminado y deberá retirarse de la colección de enemigos del escenario.

4. Enemigos

Cada enemigo deberá almacenar, como mínimo:

- Un identificador que determine su tipo.
- Su posición dentro del escenario.

Los tipos de enemigos disponibles inicialmente son:

```
TORTUGA_ROJA
TORTUGA_VERDE
GUMBA
VOCHO
PLANTA
```

El diseño deberá permitir incorporar nuevos tipos de enemigos posteriormente sin modificar innecesariamente las clases existentes.

5. Archivo de enemigos

La información de los enemigos presentes en un escenario se obtendrá de un **archivo de texto**. Cada línea del archivo representará un enemigo utilizando el siguiente formato:

```
id, posicion
```

Por ejemplo:

```
TORTUGA_ROJA , 5
GUMBA , 8
PLANTA , 12
TORTUGA_VERDE , 15
VOCHO , 21
```

Una vez creado el escenario, el programa deberá leer el archivo, crear los objetos correspondientes y almacenarlos en la estructura o colección seleccionada para representar a los enemigos.

La implementación deberá verificar que las posiciones obtenidas del archivo correspondan a posiciones válidas dentro del escenario.

El archivo no garantiza que los enemigos se encuentren ordenados de acuerdo con su posición en el escenario.

Por ejemplo, un archivo válido podría contener:

```
PLANTA ,12
GUMBA ,3
VOCHO ,20
TORTUGA_ROJA ,7
TORTUGA_VERDE ,15
```

6. Comportamiento del escenario

La clase encargada de representar el escenario deberá proporcionar, como mínimo, los siguientes métodos:

```
moverPersonaje()
actualizarListaDeEnemigos()
dibujarEscenario()
```

El método `moverPersonaje()` deberá modificar la posición actual del personaje de acuerdo con las reglas de movimiento establecidas.

El método `actualizarListaDeEnemigos()` deberá determinar qué enemigos han sido alcanzados o rebasados por el personaje y eliminarlos de la colección correspondiente.

El método `dibujarEscenario()` deberá mostrar una representación del estado actual del escenario que permita identificar, al menos, la posición del personaje y las posiciones ocupadas por los enemigos.

7. Requisitos de diseño

La solución deberá desarrollarse utilizando **Java y el paradigma orientado a objetos**. El diseño deberá identificar las clases, enumeraciones e interfaces que resulten necesarias para representar adecuadamente el problema.

Deberá seleccionarse una **estructura de datos apropiada para almacenar los enemigos**, considerando las operaciones que se realizarán sobre ellos durante el avance del personaje.

La elección de la estructura de datos deberá estar **justificada** considerando, al menos:

- La forma en que se incorporan los enemigos al escenario.
- La forma en que se determina cuál es el siguiente enemigo que encontrará el personaje.
- La eliminación de enemigos conforme avanza el personaje.
- La complejidad temporal de las principales operaciones realizadas sobre la estructura seleccionada.

La solución deberá favorecer el **encapsulamiento**, la **separación de responsabilidades** y la posibilidad de extender el sistema posteriormente con nuevos personajes y tipos de enemigos.